



Université Cheikh Anta DIOP de Dakar
Faculté des Sciences et Techniques
Département Mathématiques et Informatique



Laboratoire d'Algèbre, de Cryptologie,
de Géométrie Algébrique et Applications
(LACGAA)

Licence 3 – Transmission de Données et Sécurité de l'Information
Option : MCS

EXAMEN BASES DE DONNÉES AVANCÉES POSTGRESQL
Plateforme de Gestion des Incidents Informatiques

Informations Générales

Présentées et soutenues par : Mlle Aminata Diallo et Aissatou Gaye
Encadrante : Dr Aminata NGOM

Année Académique 2025–2026

« Des données plus structurées aujourd'hui pour des décisions plus intelligentes demain. »

Chapitre 1

Introduction

L'informatique occupe aujourd'hui une place centrale dans le fonctionnement des entreprises et des administrations : dès qu'un ordinateur, une imprimante, un logiciel ou un réseau tombe en panne, l'activité peut être ralentie ou bloquée. Pour limiter cet impact, les organisations mettent en place un service informatique chargé de recevoir les signalements de panne (appelés « incidents »), de les confier à un technicien, puis de suivre leur résolution jusqu'à la clôture. Dans la pratique, ce suivi est souvent fait de manière informelle (appels téléphoniques, messages, tableurs), ce qui pose un problème concret : on perd la trace de qui a signalé quoi, qui s'en est occupé, combien de temps cela a pris, et si le problème a réellement été résolu. C'est cette problématique – *comment centraliser, sécuriser et tracer de bout en bout le traitement des incidents informatiques d'une organisation ?* – que ce projet cherche à résoudre. Ce travail s'inscrit dans le module Bases de Données Avancées PostgreSQL, dont l'objectif n'est pas seulement de produire une application qui fonctionne, mais surtout de démontrer la maîtrise des mécanismes internes avancés de PostgreSQL (contraintes, vues, fonctions, procédures, triggers, transactions, rôles et privilèges, audit) tels qu'ils sont utilisés dans un contexte professionnel réel. Ce sujet nous propose de répondre à cette problématique par une plateforme de gestion des tickets d'incident, des affectations aux techniciens et du suivi des interventions. Concrètement, la solution repose sur deux briques complémentaires : une base de données PostgreSQL qui porte toute la logique et la sécurité des données, et une interface web développée avec Flask qui permet à chaque utilisateur (selon son rôle) d'interagir simplement avec cette base sans avoir à écrire de SQL.

Chapitre 2

Objectifs du projet

Nous avons mené ce projet pour répondre concrètement à la problématique posée, en mobilisant à chaque étape un outil ou une technique choisi pour une raison précise.

1. **Modélisation des données** : avant d'écrire la moindre table, nous avons modélisé les données avec la méthode Merise, en utilisant le logiciel Looping pour construire le Modèle Conceptuel de Données (MCD). Cet outil permet de dessiner visuellement les entités et leurs relations, et de vérifier la cohérence du modèle avant de le traduire en tables PostgreSQL.
2. **Création d'une base de données relationnelle PostgreSQL** : nous avons ensuite traduit ce MCD en tables réelles dans PostgreSQL, choisi pour sa robustesse et son support avancé des mécanismes exigés par le module (fonctions, triggers, rôles).
3. **Contraintes d'intégrité** : mises en place pour empêcher qu'une donnée incohérente soit enregistrée.
4. **Vues et requêtes SQL avancées** : créées pour éviter de réécrire les mêmes requêtes complexes à chaque besoin de statistique ou de liste d'incidents.
5. **Fonctions et procédures stockées PL/pgSQL** : écrites directement dans PostgreSQL plutôt que dispersées dans le code Flask, pour que la même règle métier s'applique quel que soit le point d'entrée.
6. **Triggers** : utilisés pour que certaines actions se déclenchent automatiquement en base, notamment l'audit.
7. **Transactions SQL** (COMMIT, ROLLBACK, SAVEPOINT) : employées pour garantir qu'un ensemble d'opérations liées soit entièrement validé ou entièrement annulé.
8. **Gestion des rôles, utilisateurs et privilèges** : trois rôles PostgreSQL distincts définis pour que chaque type d'utilisateur n'ait accès qu'aux données et actions qui le concernent.
9. **Audit et traçabilité** : mis en place pour répondre à la question « qui a fait quoi, et quand ? ».
10. **Sécurisation des accès aux données** : mots de passe hachés, contrôle applicatif et contrôle PostgreSQL combinés.
11. **Interface applicative Flask** : développée pour que tout ce qui précède reste utilisable via de simples pages web.

Chapitre 3

Architecture générale de la solution

Choix technologiques

Le choix du système de gestion de base de données s'est porté sur PostgreSQL. Contrairement à un simple fichier ou à un tableur, qui ne garantissent rien sur la cohérence des données saisies, un Système de Gestion de Base de Données (SGBD) relationnel impose lui-même des règles – contraintes, transactions, droits d'accès – qui assurent que les données restent cohérentes, y compris lorsque plusieurs utilisateurs travaillent simultanément. Par rapport à d'autres SGBD tels que MySQL, PostgreSQL a été retenu pour son support plus riche des fonctions et procédures PL/pgSQL, des triggers et de la gestion fine des rôles, ce qui correspond directement aux compétences visées par le module. Concernant la couche applicative, notre choix s'est porté sur Flask, un micro-framework Python. À la différence d'un framework plus complet comme Django, Flask ne fournit que l'essentiel – recevoir une requête web, exécuter du code, renvoyer une page – et laisse le développeur assembler le reste. Ce choix nous a permis de rester concentrés sur la logique métier et sur PostgreSQL, tout en travaillant avec un outil réellement utilisé en entreprise.

Organisation en trois couches

1. **Couche données** : PostgreSQL – tables, contraintes, vues, fonctions/procédures, triggers, rôles et privilèges.
2. **Couche applicative** : Flask (Python), organisée en trois Blueprints (`auth`, `incidents`, `admin`), avec `psycopg2` comme pilote d'accès à PostgreSQL.
3. **Couche présentation** : templates Jinja2 (HTML) et une feuille de style CSS commune.

Arborescence du projet

```
gestion_des_incidents_informatiques/  
|-- run.py Point d'entree de l'application  
|-- config.py Configuration (variables d'environnement)  
|-- .env Variables sensibles (mots de passe, cles)  
|-- requirements.txt Dependances Python  
|-- script_examen_BDD.sql Script complet de creation de la base  
|-- sql/  
| |-- roles_privileges.sql Creation des roles et privileges
```

```
| +-- transactions_demo.sql Demonstration COMMIT/ROLLBACK/SAVEPOINT
+-- app/
|-- __init__.py Fabrique de l'application (create_app)
|-- auth.py Blueprint authentication (login/logout)
|-- incidents.py Blueprint metier (dashboard, incidents, audit)
|-- admin.py Blueprint administration (utilisateurs)
|-- db.py Connexion PostgreSQL + SET ROLE
|-- helpers.py Decorateurs de securite (login/roles)
|-- static/css/style.css
+-- templates/ Pages HTML (Jinja2)
```

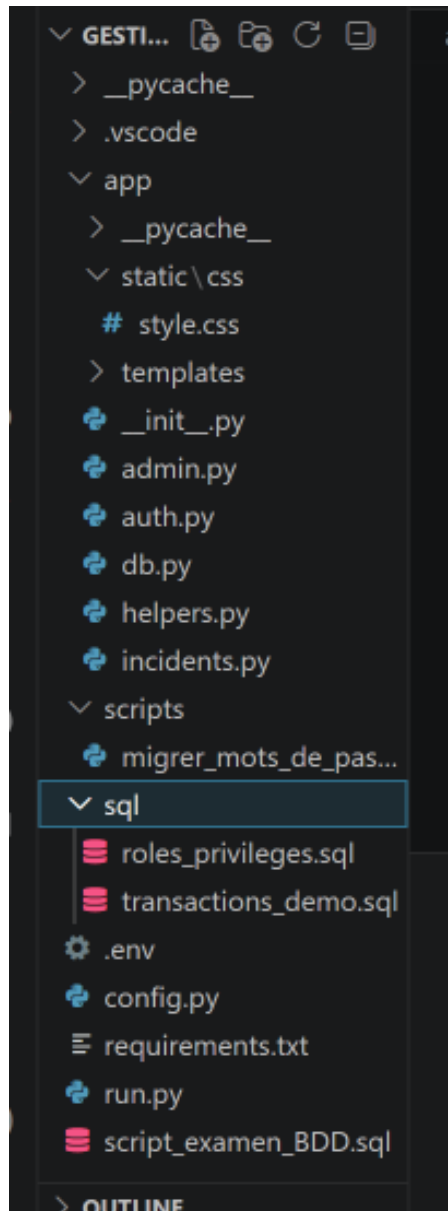


FIGURE 3.1 – Organisation des fichiers du projet dans Visual Studio Code.

Principe de sécurisation applicative + base de données

Le point clé de l'architecture est que la sécurité n'est pas gérée seulement côté Flask : chaque utilisateur connecté voit la connexion PostgreSQL basculer automatiquement sur son

rôle métier via l'instruction `SET ROLE` (fichier `app/db.py`). Ainsi, même si une faille existait côté applicatif, PostgreSQL refuserait de toute façon les opérations non autorisées grâce aux privilèges (`GRANT`) attribués à chaque rôle.

Listing 3.1 – Extrait de `app/db.py`

```
role_pg = session.get('role_pg')
if role_pg:
    with g.db.cursor() as cur:
        cur.execute(f"SET_ROLE_{role_pg};")
    g.db.commit()
```

Chapitre 4

Modélisation des données

Avant de créer la moindre table dans PostgreSQL, nous avons construit le Modèle Conceptuel de Données (MCD) du système avec le logiciel Looping, en suivant la méthode Merise. Ce modèle fait apparaître huit entités et les associations qui les relient.

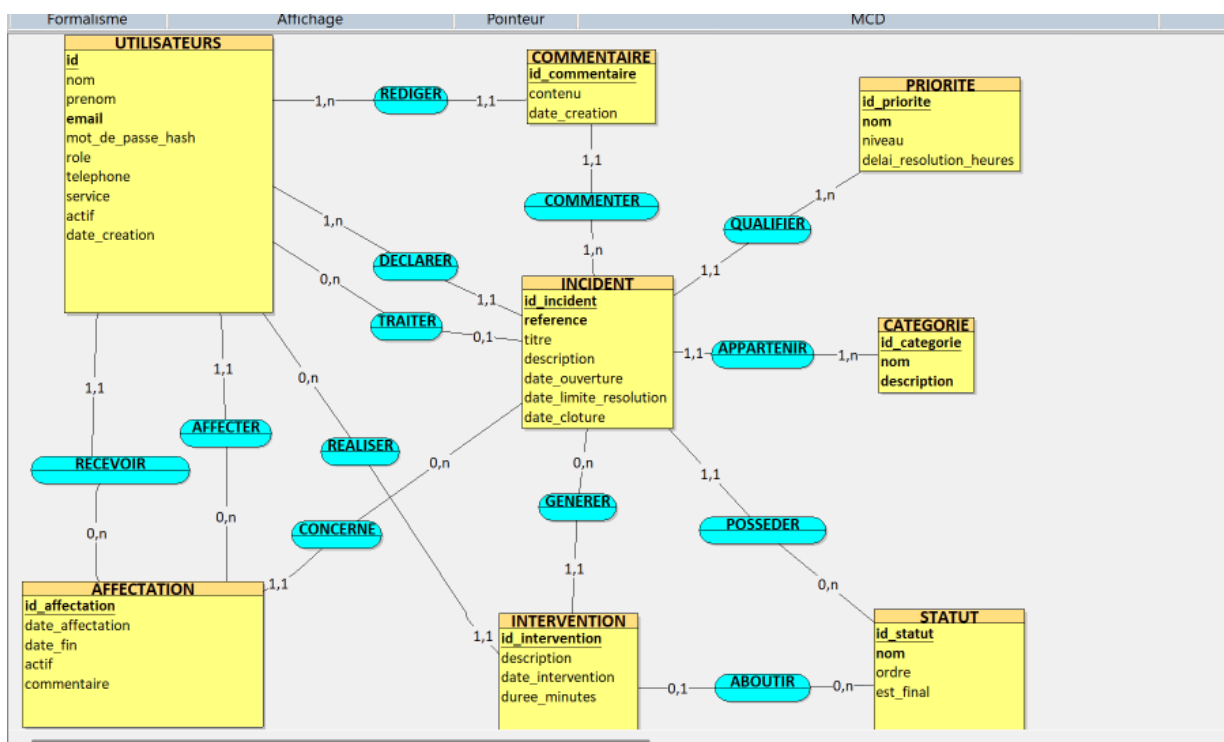


FIGURE 4.1 – Modèle Conceptuel de Données (MCD) réalisé avec Looping.

Les entités principales

1. **UTILISATEURS** centralise les comptes de la plateforme. C'est le pivot du système : c'est elle qui déclare les incidents, les commente, les traite et intervient dessus.
2. **INCIDENT** est l'entité centrale : elle porte la référence, le titre, la description, la date d'ouverture, la date limite de résolution et la date de clôture d'un ticket.
3. **PRIORITE**, **CATEGORIE** et **STATUT** sont des entités de référence, évitant de saisir en dur des valeurs comme « urgent » ou « en cours ».
4. **COMMENTAIRE**, **AFFECTATION** et **INTERVENTION** sont des entités associatives matérialisant l'historique d'un incident dans le temps.

Choix de modélisation

Le point commun de COMMENTAIRE, AFFECTATION et INTERVENTION est qu'il s'agit, conceptuellement, de relations plusieurs-à-plusieurs entre UTILISATEURS et INCIDENT. Nous avons choisi de transformer chacune de ces relations en une entité à part entière plutôt qu'en une simple table de jointure, afin de pouvoir y accrocher des informations propres à chaque occurrence (date, durée, commentaire, description) : c'est ce qui permet de reconstituer l'historique complet d'un incident plutôt qu'un simple état final.

Chapitre 5

Création de la base de données et contraintes d'intégrité

Création de la base

Le modèle conceptuel a été traduit en un schéma relationnel de neuf tables, créées à partir du script `script_examen_BDD.sql` :

```
CREATE DATABASE gestion_des_incidents_informatiques;  
psql -U postgres -d gestion_des_incidents_informatiques -f  
script_examen_BDD.sql
```

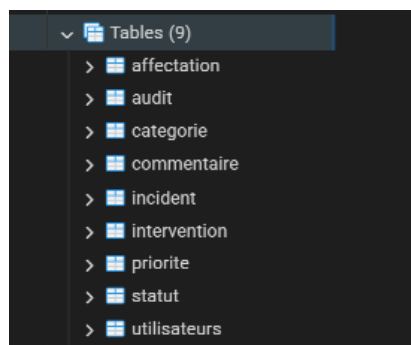


FIGURE 5.1 – Les neuf tables de la base, visibles dans pgAdmin.

Explication détaillée des contraintes, table par table

Pour chaque table, PostgreSQL permet d'imposer des règles au moment même de la création, plutôt que de les vérifier a posteriori dans le code de l'application. Ainsi, même si l'application changeait complètement, la base refuserait toujours les données incohérentes.

Table utilisateurs

```
-- DROP TABLE IF EXISTS public.utilisateurs;

CREATE TABLE IF NOT EXISTS public.utilisateurs
(
    id_utilisateur serial NOT NULL,
    nom character varying(50) COLLATE pg_catalog."default" NOT NULL,
    prenom character varying(50) COLLATE pg_catalog."default",
    email character varying(100) COLLATE pg_catalog."default" NOT NULL,
    mot_de_passe_hash text COLLATE pg_catalog."default" NOT NULL,
    role character varying(20) COLLATE pg_catalog."default",
    telephone character varying(20) COLLATE pg_catalog."default",
    service character varying(100) COLLATE pg_catalog."default",
    actif boolean DEFAULT true,
    date_creation timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT utilisateurs_pkey PRIMARY KEY (id_utilisateur),
    CONSTRAINT utilisateurs_email_key UNIQUE (email),
    CONSTRAINT utilisateurs_role_check CHECK (role::text = ANY (ARRAY['ADMIN'::character varying::text,
)
```

FIGURE 5.2 – Définition complète de la table utilisateurs.

1. `id_utilisateur serial NOT NULL` : `serial` génère automatiquement un entier incrémental (1, 2, 3...) à chaque insertion.
2. `CONSTRAINT utilisateurs_pkey PRIMARY KEY` : déclare l'identifiant unique de chaque ligne, jamais vide, jamais dupliqué.
3. `CONSTRAINT utilisateurs_email_key UNIQUE (email)` : interdit que deux comptes aient le même email.
4. `mot_de_passe_hash text NOT NULL` : stocke uniquement la version hachée du mot de passe, jamais en clair.
5. `CONSTRAINT utilisateurs_role_check CHECK (role IN ('ADMIN', 'TECHNICIEN', 'UTILISATEUR'))` : interdit d'enregistrer une valeur de rôle différente de ces trois chaînes exactes, protégeant toute la logique de privilèges du chapitre 10.
6. `actif boolean DEFAULT true` et `date_creation timestamp DEFAULT CURRENT_TIMESTAMP` : valeurs automatiques si l'application n'en précise pas.

Table incident

```
CREATE TABLE IF NOT EXISTS public.incident
(
    id_incident serial NOT NULL,
    reference character varying(30) COLLATE pg_catalog."default" NOT NULL,
    titre character varying(200) COLLATE pg_catalog."default" NOT NULL,
    description text COLLATE pg_catalog."default",
    date_ouverture timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    date_limite_resolution timestamp without time zone,
    date_cloture timestamp without time zone,
    id_utilisateur integer NOT NULL,
    id_priorite integer,
    id_statut integer,
    id_categorie integer,
    CONSTRAINT incident_pkey PRIMARY KEY (id_incident),
    CONSTRAINT incident_reference_key UNIQUE (reference),
    CONSTRAINT fk_incident_categorie FOREIGN KEY (id_categorie)
        REFERENCES public.categorie (id_categorie) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT fk_incident_priorite FOREIGN KEY (id_priorite)
        REFERENCES public.priorite (id_priorite) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT fk_incident_statut FOREIGN KEY (id_statut)
```

FIGURE 5.3 – Définition de la table incident (colonnes et clés étrangères).

```
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT fk_incident_user FOREIGN KEY (id_utilisateur)
        REFERENCES public.utilisateurs (id_utilisateur) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
)
```

FIGURE 5.4 – Suite : contrainte fk_incident_user.

C'est la table qui illustre le mieux la contrainte de clé étrangère (FOREIGN KEY). `fk_incident_user` garantit qu'un incident ne peut être créé que si `id_utilisateur` correspond à un utilisateur qui existe réellement. De la même manière, `fk_incident_priorite`, `fk_incident_statut` et `fk_incident_categorie` garantissent qu'un incident référence toujours une priorité, un statut et une catégorie réels – c'est exactement cette règle qui a été testée en section 9 avec un `id_priorite` invalide (999999), automatiquement rejeté par PostgreSQL.

Synthèse des contraintes utilisées

Type de contrainte	Ce qu'elle garantit	Exemples dans le projet
PRIMARY KEY	Identifiant unique et jamais vide	<code>id_incident</code> , <code>id_utilisateur</code> , <code>id_audit...</code>
FOREIGN KEY	Ne référencer qu'une valeur qui existe réellement	<code>fk_incident_user</code> , <code>affectation_id_incident_fkey...</code>
UNIQUE	Aucune duplication possible	<code>utilisateurs.email</code> , <code>incident.reference</code> , <code>categorie.nom</code>
NOT NULL	Un champ ne peut jamais rester vide	<code>incident.titre</code> , <code>utilisateurs.mot_de_passe_hash</code>
CHECK	Seules certaines valeurs précises sont acceptées	<code>utilisateurs_role_check</code> (ADMIN/TECHNICIEN/UTILISATEUR)
DEFAULT	Valeur automatique si rien n'est précisé	<code>actif = true</code> , <code>date_ouverture = CURRENT_TIMESTAMP</code>

TABLE 5.1 – Synthèse des contraintes d'intégrité utilisées.

Chapitre 6

Vues SQL

Une vue est une requête SQL enregistrée sous un nom, comme si c'était une table. PostgreSQL ne stocke aucune donnée propre à une vue : à chaque appel, il ré-exécute la requête définie derrière. L'intérêt est double : simplifier le code applicatif (une seule instruction **SELECT** plutôt qu'une jointure complexe répétée), et centraliser la logique (modifier la vue une seule fois en base plutôt que dans plusieurs fichiers).

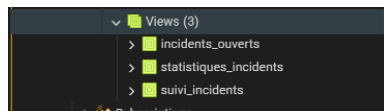


FIGURE 6.1 – Les trois vues créées, visibles dans pgAdmin.

incidents_ouverts

```
CREATE OR REPLACE VIEW public.incidents_ouverts
AS
SELECT i.id_incident,
       i.reference,
       i.titre,
       i.description,
       i.date_ouverture,
       s.nom AS statut,
       p.nom AS priorite
FROM incident i
      JOIN statut s ON i.id_statut = s.id_statut
      JOIN priorite p ON i.id_priorite = p.id_priorite
WHERE s.nom::text = 'OUVERT'::text;

ALTER TABLE public.incidents_ouverts
  OWNER TO postgres;
```

FIGURE 6.2 – Définition de la vue incidents_ouverts.

Cette vue relie `incident` à `statut` et `priorite` par `JOIN`, puis ne garde (clause `WHERE`) que les incidents dont le statut porte le nom `'OUVERT'`. Elle alimente le tableau de bord.

statistiques_incidents

```
CREATE OR REPLACE VIEW public.statistiques_incidents
AS
SELECT s.nom AS statut,
       count(i.id_incident) AS nombre_incidents
FROM incident i
      JOIN statut s ON i.id_statut = s.id_statut
GROUP BY s.nom;

ALTER TABLE public.statistiques_incidents
OWNER TO postgres;
```

FIGURE 6.3 – Définition de la vue statistiques_incidents.

Elle utilise GROUP BY pour regrouper les incidents par nom de statut, et count() pour compter combien d'incidents appartiennent à chaque groupe. Le résultat alimente le tableau « Statistiques par statut » du tableau de bord.

suivi_incidents

```
Query History
CREATE OR REPLACE VIEW public.suivi_incidents
AS
SELECT i.reference,
       i.titre,
       i.description,
       (u.nom::text || ' '::text) || u.prenom::text AS utilisateur,
       c.nom AS categorie,
       p.nom AS priorite,
       s.nom AS statut,
       i.date_ouverture
FROM incident i
      JOIN utilisateurs u ON i.id_utilisateur = u.id_utilisateur
      JOIN categorie c ON i.id_categorie = c.id_categorie
      JOIN priorite p ON i.id_priorite = p.id_priorite
      JOIN statut s ON i.id_statut = s.id_statut;

ALTER TABLE public.suivi_incidents
OWNER TO postgres;
```

FIGURE 6.4 – Définition de la vue suivi_incidents.

Cette vue combine quatre tables en une seule ligne de résultat par incident. L'opérateur || concatène le nom et le prénom du déclarant en une seule chaîne. Elle sert de base aux rapports de suivi complets.

Chapitre 7

Fonctions et procédures PL/pgSQL

PL/pgSQL permet d'écrire de la logique directement à l'intérieur de PostgreSQL, sous forme de blocs nommés. Une **FONCTION** renvoie toujours une valeur et s'appelle avec **SELECT** ; une **PROCEDURE** ne renvoie jamais de valeur et s'appelle avec **CALL**. Six fonctions/procédures ont été créées.

creer_incident (FONCTION)

```
CREATE OR REPLACE FUNCTION public.creer_incident(  
    p_reference character varying,  
    p_titre character varying,  
    p_description text,  
    p_utilisateur integer,  
    p_priorite integer,  
    p_statut integer,  
    p_categorie integer)  
    RETURNS void  
    LANGUAGE 'plpgsql'  
    COST 100  
    VOLATILE PARALLEL UNSAFE  
AS $BODY$  
  
BEGIN  
  
INSERT INTO incident  
(  
    reference,  
    titre,  
    description,  
    id_utilisateur,  
    id_priorite,
```

FIGURE 7.1 – Définition de `creer_incident` (début).

```

id_statut,
id_categorie
)

VALUES
(
p_reference,
p_titre,
p_description,
p_utilisateur,
p_priorite,
p_statut,
p_categorie
);

END;

$BODY$;

ALTER FUNCTION public.creer_incident(character varying, character varying, text, integer, integer, integer, integer)
OWNER TO postgres;

GRANT EXECUTE ON FUNCTION public.creer_incident(character varying, character varying, text, integer, integer, integer, integer) TO

```

FIGURE 7.2 – Suite : corps et GRANT EXECUTE.

Insère un nouvel incident. RETURNS void indique qu'elle ne calcule pas de résultat. COST 100 aide l'optimiseur de requêtes ; VOLATILE indique qu'elle peut modifier la base (par opposition à IMMUTABLE/STABLE) ; PARALLEL UNSAFE interdit toute exécution parallèle, nécessaire dès qu'une fonction écrit des données. GRANT EXECUTE accorde le droit de l'appeler à des rôles précis (chapitre 10).

affecter_technicien (FONCTION)

```

CREATE OR REPLACE FUNCTION public.affecter_technicien(
    p_technicien integer,
    p_incident integer,
    p_commentaire text)
    RETURNS void
    LANGUAGE 'plpgsql'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$

BEGIN

INSERT INTO affectation
(
id_utilisateur,
id_incident,
commentaire
)

```

FIGURE 7.3 – Définition de affecter_technicien.

Enregistre qu'un technicien est affecté à un incident, via un INSERT dans affectation – chaque appel ajoute une ligne à l'historique, sans écraser les affectations précédentes.

changer_statut_incident (FONCTION)

```
CREATE OR REPLACE FUNCTION public.changer_statut_incident(  
    p_incident integer,  
    p_nouveau_statut integer)  
RETURNS void  
LANGUAGE 'plpgsql'  
COST 100  
VOLATILE PARALLEL UNSAFE  
AS $BODY$  
  
BEGIN  
  
UPDATE incident  
  
SET id_statut = p_nouveau_statut  
  
WHERE id_incident = p_incident;  
  
END;  
  
$BODY$;  
  
ALTER FUNCTION public.changer_statut_incident(integer, integer)  
OWNER TO postgres;
```

FIGURE 7.4 – Définition de changer_statut_incident.

Simple UPDATE ciblé par WHERE id_incident = p_incident. Déclenche automatiquement le trigger trg_cloture_incident (section 8).

ajouter_commentaire (PROCEDURE)

```
CREATE OR REPLACE PROCEDURE public.ajouter_commentaire(  
    IN p_contenu text,  
    IN p_utilisateur integer,  
    IN p_incident integer)  
LANGUAGE 'plpgsql'  
AS $BODY$  
  
BEGIN  
  
    INSERT INTO commentaire  
    (  
        contenu,  
        id_utilisateur,  
        id_incident  
    )  
  
    VALUES  
    (  
        p_contenu,  
        p_utilisateur,  
        p_incident  
    );  
  
END;
```

FIGURE 7.5 – Définition de ajouter_commentaire.

IN précise un paramètre d'entrée. Contrairement à une fonction, aucun RETURNS : appelée avec CALL, jamais SELECT.

ajouter_intervention (PROCEDURE)

```
CREATE OR REPLACE PROCEDURE public.ajouter_intervention(  
    IN p_description text,  
    IN p_duree integer,  
    IN p_incident integer,  
    IN p_statut integer)  
LANGUAGE 'plpgsql'  
AS $BODY$  
  
BEGIN  
  
    INSERT INTO intervention  
    (  
        description,  
        duree_minutes,  
        id_incident,  
        id_statut  
    )  
  
    VALUES  
    (  
        p_description,  
        p_duree,  
        p_incident,  
        p_statut  
    )  
;
```

FIGURE 7.6 – Définition de ajouter_intervention.

Enregistre une intervention technique ; chaque appel crée une nouvelle ligne dans `intervention`, préservant l'historique complet.

cloturer_incident (PROCEDURE)

```
CREATE OR REPLACE PROCEDURE public.cloturer_incident(  
    IN p_id_incident integer)  
LANGUAGE 'plpgsql'  
AS $BODY$  
  
BEGIN  
  
    UPDATE incident  
  
    SET  
        id_statut = (  
            SELECT id_statut  
            FROM statut  
            WHERE nom = 'FERME'  
        ),  
        date_cloture = CURRENT_TIMESTAMP  
  
    WHERE id_incident = p_id_incident;  
  
END;  
  
$BODY$;  
ALTER PROCEDURE public.cloturer_incident(integer)  
    OWNER TO postgres;
```

FIGURE 7.7 – Définition de cloturer_incident.

La sous-requête (`SELECT id_statut FROM statut WHERE nom = 'FERME'`) va chercher dynamiquement l'identifiant du statut par son nom plutôt que de l'écrire en dur, rendant le code robuste. `CURRENT_TIMESTAMP` fixe la date de clôture.

Chapitre 8

Triggers (déclencheurs)

Un trigger exécute automatiquement une fonction PL/pgSQL quand un événement précis se produit sur une table (INSERT, UPDATE ou DELETE), sans que l'application n'ait rien à demander. Trois triggers ont été créés sur la table `incident`.

trg_audit_incident – tracer automatiquement toute modification

```
CREATE OR REPLACE FUNCTION public.fn_audit_incident()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO audit(operation, table_nom, nouvelle_valeur, utilisateur)
    VALUES ('INSERT', 'incident', row_to_json(NEW)::text, current_user);
    RETURN NEW;

  ELSIF TG_OP = 'UPDATE' THEN
    INSERT INTO audit(operation, table_nom, ancienne_valeur, nouvelle_valeur, utilisateur)
    VALUES ('UPDATE', 'incident', row_to_json(OLD)::text, row_to_json(NEW)::text, current_user);
    RETURN NEW;

  ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO audit(operation, table_nom, ancienne_valeur, utilisateur)
    VALUES ('DELETE', 'incident', row_to_json(OLD)::text, current_user);
    RETURN OLD;

  END IF;
```

FIGURE 8.1 – Fonction `fn_audit_incident` (IF `TG_OP...`).

`TG_OP` est une variable spéciale contenant le type d'opération en cours ('INSERT', 'UPDATE' ou 'DELETE'). `NEW` et `OLD` représentent respectivement la ligne après et avant modification. `row_to_json(NEW)::text` convertit toute la ligne en JSON pour la stocker dans `audit`. `current_user` renvoie le rôle PostgreSQL actif au moment de l'action, reflet du mécanisme `SET ROLE` (section 10). `AFTER INSERT OR DELETE OR UPDATE ON incident FOR EACH ROW` déclenche la fonction une fois par ligne affectée, après l'opération.

trg_cloture_incident – dater automatiquement la clôture

```
CREATE OR REPLACE FUNCTION public.fn_cloture_incident()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
BEGIN

  IF NEW.id_statut IS NOT NULL THEN

    IF EXISTS (
      SELECT 1 FROM statut
      WHERE id_statut = NEW.id_statut
      AND est_final = true
    ) THEN
      NEW.date_cloture = CURRENT_TIMESTAMP;
    END IF;

  END IF;

  RETURN NEW;
END;
$BODY$;
```

FIGURE 8.2 – Fonction fn_cloture_incident.

BEFORE UPDATE : le trigger agit avant l'écriture, ce qui est indispensable puisqu'il modifie NEW.date_cloture directement. IF EXISTS (...) vérifie simplement si un statut marqué comme final correspond, sans récupérer la ligne entière.

trg_verif_incident – empêcher la création d’un incident incomplet

```
CREATE OR REPLACE FUNCTION public.fn_verif_incident()
    RETURNS trigger
    LANGUAGE 'plpgsql'
    COST 100
    VOLATILE NOT LEAKPROOF
AS $BODY$
BEGIN

    IF NEW.id_priorite IS NULL THEN
        RAISE EXCEPTION 'Priorité obligatoire pour créer un incident';
    END IF;

    IF NEW.id_statut IS NULL THEN
        RAISE EXCEPTION 'Statut obligatoire pour créer un incident';
    END IF;

    RETURN NEW;
END;
$BODY$;

ALTER FUNCTION public.fn_verif_incident()
    OWNER TO postgres;
```

FIGURE 8.3 – Fonction fn_verif_incident.

BEFORE INSERT : bloque l’insertion si la priorité ou le statut sont manquants, avec RAISE EXCEPTION qui interrompt immédiatement l’exécution et fait échouer la transaction en cours.

Chapitre 9

Transactions complexes (COMMIT / ROLLBACK / SAVEPOINT)

Une transaction regroupe plusieurs instructions SQL en une seule unité « tout ou rien » : soit toutes les instructions sont validées ensemble (COMMIT), soit aucune ne l'est (ROLLBACK). Trois scénarios illustrent ce mécanisme.

Transaction 1 – création d'incident atomique

```
-----  
-- ● TRANSACTION 1 : Création incident + affectation + commentaire  
-----  
  
BEGIN;  
  
-- Création de l'incident  
SELECT public.creer_incident(  
    'INC-DEMO-001',  
    'Imprimante RH hors service',  
    'Aucune impression possible depuis ce matin',  
    (SELECT id_utilisateur FROM utilisateurs WHERE role = 'UTILISATEUR' LIMIT 1),  
    (SELECT id_priorite FROM priorite ORDER BY niveau LIMIT 1),  
    (SELECT id_statut FROM statut ORDER BY ordre LIMIT 1),  
    (SELECT id_categorie FROM categorie LIMIT 1)  
);  
  
-- Affectation du technicien  
SELECT public.affecter_technicien(  
    (SELECT id_utilisateur FROM utilisateurs WHERE role = 'TECHNICIEN' LIMIT 1),  
    (SELECT id_incident FROM incident WHERE reference = 'INC-DEMO-001'),  
    'Affectation automatique a la creation'  
);  
  
-- Ajout commentaire via procédure (CALL)  
DO $$  
DECLARE  
    v_id_technicien INTEGER;  
    v_id_incident INTEGER;
```

FIGURE 9.1 – Création incident + affectation (début de la transaction).


```

DO $$
DECLARE
    v_id_technicien INTEGER;
    v_id_incident INTEGER;
BEGIN
    SELECT id_utilisateur INTO v_id_technicien
    FROM utilisateurs
    WHERE role = 'TECHNICIEN'
    LIMIT 1;

    SELECT id_incident INTO v_id_incident
    FROM incident
    WHERE reference = 'INC-DEMO-001';

    CALL public.ajouter_commentaire(
        'Ticket cree et affecte, prise en charge en cours.',
        v_id_technicien,
        v_id_incident
    );
END $$;

COMMIT;

```

FIGURE 9.2 – Ajout du commentaire via bloc anonyme puis COMMIT.

Cette transaction regroupe trois actions liées – créer l’incident, l’affecter, ajouter un commentaire – validées ensemble avec COMMIT. Le bloc DO \$\$... \$\$ est un bloc anonyme : il permet d’exécuter une suite d’instructions PL/pgSQL avec variables (DECLARE) directement depuis une requête, sans créer de fonction permanente. SELECT ... INTO v_variable place le résultat d’une requête dans une variable, nécessaire ici car CALL ne peut pas recevoir de sous-requêtes en paramètre.

Transaction 2 – annulation ciblée avec SAVEPOINT

```
-----  
-- 🟡 TRANSACTION 2 : Intervention + clôture + SAVEPOINT  
-----  
  
BEGIN;  
  
-- Intervention initiale  
DO $$  
DECLARE  
    v_id_incident INTEGER;  
    v_id_statut INTEGER;  
BEGIN  
    SELECT id_incident INTO v_id_incident  
    FROM incident  
    WHERE reference = 'INC-DEMO-001';  
  
    SELECT id_statut INTO v_id_statut  
    FROM statut  
    ORDER BY ordre  
    LIMIT 1;  
  
    CALL public.ajouter_intervention(  
        'Redemarrage du service impression',  
        30,  
        v_id_incident,  
        v_id_statut  
    );  
END $$;
```

FIGURE 9.3 – Intervention initiale.

```

-- Clôture incident
DO $$
DECLARE
    v_id_incident INTEGER;
BEGIN
    SELECT id_incident INTO v_id_incident
    FROM incident
    WHERE reference = 'INC-DEMO-001';

    CALL public.cloturer_incident(v_id_incident);
END $$;

SAVEPOINT apres_cloture;

-- Intervention de test (sera annulée)
DO $$
DECLARE
    v_id_incident INTEGER;
    v_id_statut INTEGER;
BEGIN
    SELECT id_incident INTO v_id_incident
    FROM incident
    WHERE reference = 'INC-DEMO-001';

    SELECT id_statut INTO v_id_statut
    FROM statut
    ORDER BY ordre
    LIMIT 1;

```

FIGURE 9.4 – Clôture puis SAVEPOINT apres_cloture.

```

    CALL public.ajouter_intervention(
        'Intervention de test',
        15,
        v_id_incident,
        v_id_statut
    );
END $$;

-- Annulation uniquement de la dernière opération
ROLLBACK TO SAVEPOINT apres_cloture;

COMMIT;

```

FIGURE 9.5 – Intervention de test puis ROLLBACK TO SAVEPOINT.

SAVEPOINT apres_cloture pose un point de sauvegarde nommé. ROLLBACK TO SAVEPOINT apres_cloture annule uniquement les instructions exécutées après ce point (l'intervention de test), tout en conservant l'intervention initiale et la clôture, exécutées avant. C'est la différence fondamentale avec un ROLLBACK simple qui aurait tout annulé depuis le BEGIN.

Transaction 3 – annulation totale en cas d’erreur

```
-- ● TRANSACTION 3 : Test de rollback (erreur volontaire FK)
-----

BEGIN;

-- Test d'erreur volontaire (clé étrangère invalide)
SELECT public.creer_incident(
    'INC-DEMO-ERREUR',
    'Test de rollback complet',
    'Ne doit jamais apparaitre en base',
    (SELECT id_utilisateur FROM utilisateurs WHERE role = 'UTILISATEUR' LIMIT 1),
    999999, -- ✗ ID invalide pour provoquer une erreur FK
    (SELECT id_statut FROM statut ORDER BY ordre LIMIT 1),
    (SELECT id_categorie FROM categorie LIMIT 1)
);

-- Transaction annulée automatiquement après erreur
ROLLBACK;
```

FIGURE 9.6 – Tentative de création avec une priorité invalide (999999) puis ROLLBACK.

La valeur 999999 ne correspond à aucune priorité existante, ce qui viole la contrainte `fk_incident_priorite` (section 5.2) et fait échouer l’instruction. `ROLLBACK` confirme l’annulation complète : ni l’incident, ni la moindre trace dans `audit` ne subsistent.

Chapitre 10

Gestion des rôles, utilisateurs et privilèges

Le principe est de ne pas se reposer uniquement sur le code Flask pour restreindre les droits d'un utilisateur : la sécurité existe au niveau de la base elle-même, de sorte que PostgreSQL refuse de lui-même toute opération non autorisée. Cinq rôles composent l'architecture de sécurité du projet.

postgres – administrateur du serveur

```
CREATE ROLE app_login WITH
LOGIN
NOSUPERUSER
INHERIT
NOCREATEDB
NOCREATEROLE
NOREPLICATION
NOBYPASSRLS
ENCRYPTED PASSWORD 'SCRAM-SHA-256$4096:h8AmrPagZQkcCcZ01M34sg==$GrExogq//QZ0TQM0aIbIQU949QGvIveIRq8Rx fNIMJ8
GRANT admin_role, technicien_role, utilisateur_role TO app_login WITH INHERIT OPTION, SET OPTION;
```

FIGURE 10.1 – Rôle postgres (superutilisateur).

Rôle superutilisateur installé par défaut, utilisé uniquement pour l'administration – jamais par l'application.

app_login – le seul point d'entrée de l'application

```
CREATE ROLE postgres WITH
LOGIN
SUPERUSER
INHERIT
CREATEDB
CREATEROLE
REPLICATION
BYPASSRLS
ENCRYPTED PASSWORD 'SCRAM-SHA-256$4096:3imMso19laGuU4oVSYBhvQ==$515mgNAKojYiJ7rGbRBNU+8hH9Bo6keMH1eDdEVbCfs
```

FIGURE 10.2 – Rôle app_login avec GRANT des 3 rôles métier.

Seul rôle avec lequel Flask se connecte à PostgreSQL. Il ne possède aucun privilège propre. La ligne `GRANT admin_role, technicien_role, utilisateur_role TO app_login WITH INHERIT`

OPTION, SET OPTION est la plus importante : SET OPTION autorise concrètement `app_login` à exécuter SET ROLE en cours de session.

Les trois rôles métier

```
CREATE ROLE admin_role WITH
NOLOGIN
NOSUPERUSER
INHERIT
NOCREATEDB
NOCREATEROLE
NOREPLICATION
NOBYPASSRLS
ENCRYPTED PASSWORD 'SCRAM-SHA-256$4096:PwH1TBRjeGM6frIGeswIJw==$UGnIeWHP2r9eespAV+fgYucjxZDUSKSB34nLFMxqu0I
```

FIGURE 10.3 – `admin_role` (NOLOGIN).

```
CREATE ROLE technicien_role WITH
NOLOGIN
NOSUPERUSER
INHERIT
NOCREATEDB
NOCREATEROLE
NOREPLICATION
NOBYPASSRLS
ENCRYPTED PASSWORD 'SCRAM-SHA-256$4096:mKlnb550KJjxFz43CLTSgA==$Nj/Xy/NDPS8ntB0DY3z0DbP2dldesaYnuq3XvBqVAJ6
```

FIGURE 10.4 – `technicien_role` (NOLOGIN).

```
CREATE ROLE utilisateur_role WITH
NOLOGIN
NOSUPERUSER
INHERIT
NOCREATEDB
NOCREATEROLE
NOREPLICATION
NOBYPASSRLS
ENCRYPTED PASSWORD 'SCRAM-SHA-256$4096:6YKBpSyzFKTq9CTEfCWJoQ==$wanVNABeWJKN/AOBx+Py5BZMkUMfpjr8Ahh6GExueR6
```

FIGURE 10.5 – `utilisateur_role` (NOLOGIN).

Les trois portent NOLOGIN : ce sont des rôles de groupe, jamais destinés à une connexion directe – on n’y accède qu’en étant membre d’un rôle habilité à faire SET ROLE vers eux, ici uniquement `app_login`.

Rôle PostgreSQL	PostgreSQL	Correspond à	Privilèges principaux
admin_role		ADMIN	Accès complet : SELECT/INSERT/UPDATE/DELETE sur toutes les tables et vues, EXECUTE sur toutes les fonctions/procédures, gestion des utilisateurs.
technicien_role		TECHNICIEN	SELECT/UPDATE sur <code>incident</code> , SELECT sur les tables de référence, INSERT+SELECT sur <code>affectation/commentaire/intervention</code> , EXECUTE sur les fonctions de traitement.
utilisateur_role		UTILISATEUR	SELECT/INSERT sur <code>incident</code> (création), SELECT sur les tables de référence, SELECT sur <code>affectation/commentaire/intervention/utilisateurs</code> (lecture), EXECUTE sur <code>creer_incident</code> et <code>ajouter_commentaire</code> uniquement.

TABLE 10.1 – Correspondance entre rôles métier et rôles PostgreSQL.

Le mécanisme complet, du login à la requête SQL

1. La personne se connecte sur `/login` avec son email et son mot de passe, vérifiés contre la table `utilisateurs` – aucun rapport avec les rôles PostgreSQL à ce stade.
2. Flask sait quel rôle applicatif (ADMIN/TECHNICIEN/UTILISATEUR) correspond à cette personne, stocké en session.
3. À chaque requête, `app/db.py` traduit ce rôle en rôle PostgreSQL correspondant (via `ROLE_MAP`) et exécute `SET ROLE`.
4. Toute requête SQL suivante est soumise aux privilèges du rôle PostgreSQL actif, pas à ceux d'`app_login` ni du superutilisateur.

Chapitre 11

Audit et traçabilité des opérations

La traçabilité repose entièrement sur la table `audit` et sur le trigger `trg_audit_incident` (section 7.1). Le point important est que la trace est créée par PostgreSQL lui-même, pas par le code Flask : même une modification faite directement en SQL est automatiquement enregistrée.

```
CREATE TABLE IF NOT EXISTS public.audit
(
    id_audit serial NOT NULL,
    operation character varying(20) COLLATE pg_catalog."default",
    table_nom character varying(50) COLLATE pg_catalog."default",
    ancienne_valeur text COLLATE pg_catalog."default",
    nouvelle_valeur text COLLATE pg_catalog."default",
    date_action timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    utilisateur character varying(100) COLLATE pg_catalog."default",
    CONSTRAINT audit_pkey PRIMARY KEY (id_audit)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.audit
    OWNER to postgres;

GRANT ALL ON TABLE public.audit TO admin_role;

GRANT ALL ON TABLE public.audit TO postgres;
```

FIGURE 11.1 – Structure de la table `audit` et ses GRANT.

Les GRANT ALL confirment que seuls `admin_role` et `postgres` ont accès à cette table, cohérent avec le fait que la page `/audit` est réservée au rôle ADMIN.

Contenu réel après quelques opérations

id_audit	operation	table_nom	date_action	utilisateur
6	UPDATE	incident	2026-07-04 01:17:32.537882	postgres
5	INSERT	incident	2026-07-04 01:02:18.333608	postgres
2	UPDATE	incident	2026-07-02 18:38:24.849966	postgres
1	INSERT	incident	2026-07-02 17:53:04.214036	postgres
(4 rows)				

FIGURE 11.2 – Contenu de la table `audit` après quelques opérations.

Ce résultat confirme que le trigger fonctionne : chaque INSERT et UPDATE sur `incident` génère automatiquement une ligne dans `audit`, avec le type d'opération, la table concernée et l'horodatage. La colonne `utilisateur` affiche `postgres` pour ces lignes car ces opérations ont été exécutées directement en SQL pendant les tests ; en usage normal via l'application, cette

colonne affiche `admin_role`, `technicien_role` ou `utilisateur_role` grâce au mécanisme SET ROLE. Grâce à cette table, il est possible de répondre a posteriori, pour n'importe quel incident, aux questions : quand l'incident a-t-il été modifié (`date_action`) ? quel type d'opération (`operation`) ? quel était l'état avant et après (`ancienne_valeur` / `nouvelle_valeur` au format JSON) ? sous quel rôle l'opération a-t-elle été réalisée (`utilisateur`) ? Cette table est directement consultable depuis l'interface via la page `/audit` (chapitre 13).

Chapitre 12

Sécurisation des accès aux données

Le hachage des mots de passe

Stocker un mot de passe en clair est une faille de sécurité majeure. Le hachage est une transformation à sens unique : impossible de retrouver le mot de passe d'origine à partir du hash, contrairement au chiffrement qui est réversible. Le projet utilise `generate_password_hash()` et `check_password_hash()` du module `werkzeug.security`, qui appliquent l'algorithme `scrypt`. Un hash généré ressemble à : `scrypt:32768:8:1$IKmkmFs0KdXM8vap$a22e9608bfb4646...` Cette chaîne se décompose en : le nom de l'algorithme, les paramètres de coût (`32768:8:1`, qui rendent le calcul volontairement lent pour ralentir une attaque par force brute), un sel aléatoire généré à chaque hachage, puis le résultat du hachage. Le sel garantit que deux utilisateurs avec le même mot de passe obtiennent des hash complètement différents en base, rendant inutile une précalculation de type « rainbow table ». À la connexion, `check_password_hash()` relit le sel et les paramètres contenus dans le hash stocké, réapplique le même calcul sur le mot de passe saisi, puis compare les deux résultats – jamais de comparaison directe avec `==`.

La bascule automatique de rôle (SET ROLE)

`role_pg` n'est jamais une donnée saisie librement par l'utilisateur : elle provient exclusivement de `ROLE_MAP`, un dictionnaire fixe défini dans `config.py`, appliqué au rôle de l'utilisateur tel qu'il a été lu en base à la connexion. Cela élimine tout risque d'injection SQL à cet endroit. Cette bascule garantit une double barrière : même si une faille permettait de contourner les vérifications Flask, PostgreSQL refuserait quand même la requête, faute de privilège pour le rôle actif.

Le contrôle d'accès côté Flask

```
def login_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if "id_utilisateur" not in session:
            flash("Merci_de_vous_connecter.", "warning")
            return redirect(url_for("auth.login"))
        return view(*args, **kwargs)
```

```

return wrapped
def roles_required(*roles_ autorises):
def decorator(view):
    @wraps(view)
def wrapped(*args , **kwargs):
if session.get("role") not in roles_ autorises:
        abort(403)
return view(*args , **kwargs)
return wrapped
return decorator

```

Un décorateur Python enveloppe une autre fonction pour lui ajouter un comportement. `login_required` vérifie la présence d'un identifiant en session avant d'atteindre le code de la route. `roles_required("ADMIN", "TECHNICIEN")` est une fabrique de décorateurs, utilisée différemment selon la route. `abort(403)` interrompt la requête avec le code HTTP signifiant « authentifié mais non autorisé ».

Protection contre l'injection SQL

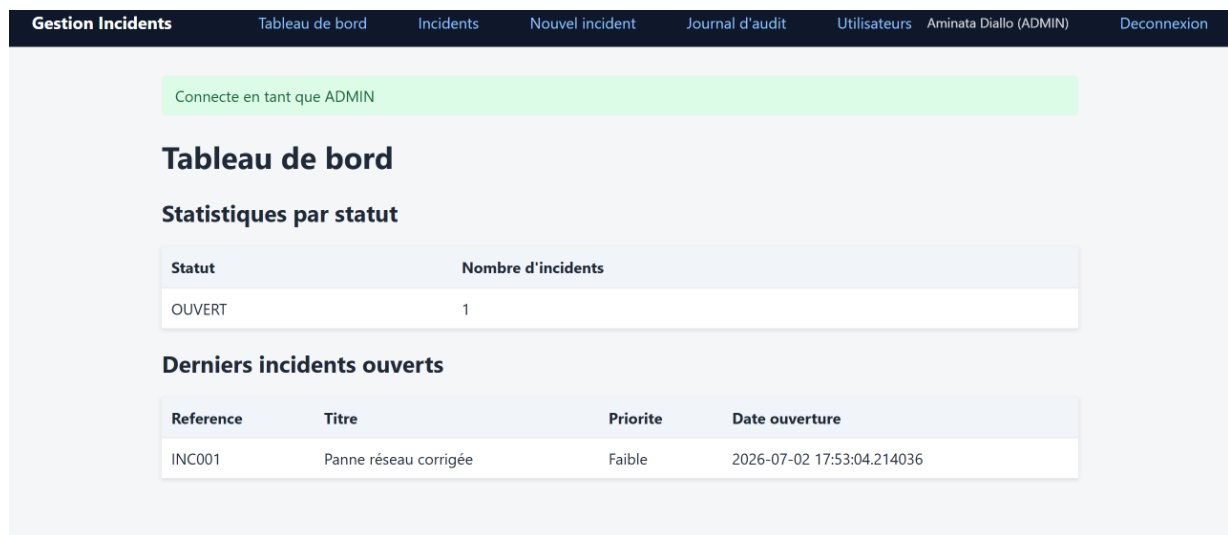
Toutes les requêtes intégrant des données saisies utilisent des requêtes paramétrées (%s) plutôt qu'une concaténation de texte. `psycpg2` envoie le texte de la requête et la valeur séparément au serveur, qui les combine lui-même de façon sûre – un texte malveillant saisi dans un formulaire est toujours traité comme une simple valeur, jamais comme du code SQL.

Chapitre 13

Interface applicative (Flask)

L'interface est organisée en pages, chacune associée à une route Flask et à un template HTML. Le menu de navigation s'adapte dynamiquement au rôle de la personne connectée.

Vue ADMIN



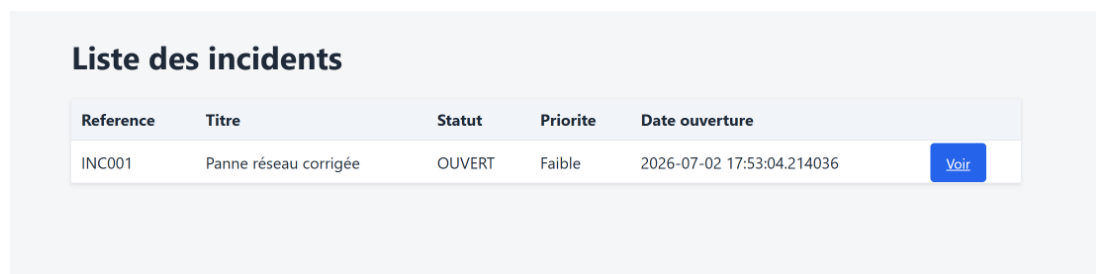
The screenshot shows the Admin Dashboard interface. At the top is a dark navigation bar with links: Gestion Incidents, Tableau de bord, Incidents, Nouvel incident, Journal d'audit, Utilisateurs, Aminata Diallo (ADMIN), and Deconnexion. Below the navigation bar, there is a green box with the text "Connecte en tant que ADMIN". The main content area is titled "Tableau de bord" and contains two sections. The first section, "Statistiques par statut", displays a table with the following data:

Statut	Nombre d'incidents
OUVERT	1

The second section, "Derniers incidents ouverts", displays a table with the following data:

Reference	Titre	Priorite	Date ouverture
INC001	Panne réseau corrigée	Faible	2026-07-02 17:53:04.214036

FIGURE 13.1 – Tableau de bord (ADMIN) : menu complet, statistiques et derniers incidents.



The screenshot shows the "Liste des incidents" page. It features a table with the following data:

Reference	Titre	Statut	Priorite	Date ouverture
INC001	Panne réseau corrigée	OUVERT	Faible	2026-07-02 17:53:04.214036

To the right of the table row, there is a blue button labeled "Voir".

FIGURE 13.2 – Liste des incidents.

INC001 - Panne réseau corrigée

Statut : OUVERT

Priorité : Faible

Categorie : Réseau

Declare par : Souleymane Ba

Ouvert le : 2026-07-02 17:53:04.214036

Description :
Internet indisponible

FIGURE 13.3 – Détail d'un incident : informations générales.

Actions technicien / admin

Affecter a un technicien

Souleymane Ba

Commentaire d'affectation

Affecter

Changer le statut

OUVERT

Mettre a jour le statut

Nouvelle intervention

FIGURE 13.4 – Bloc Actions technicien/admin (affecter, changer statut).

Nouvelle intervention

Duree (minutes)

Statut apres intervention

OUVERT

Enregistrer l'intervention

Cloturer l'incident

FIGURE 13.5 – Formulaire d'intervention et bouton de clôture.

Commentaires

Auteur	Date	Contenu
Aucun commentaire.		

Ajouter un commentaire

Envoyer

Interventions

Date	Duree (min)	Description
2026-07-02 18:06:45.672286	45	Problème résolu après remplacement du routeur

FIGURE 13.6 – Historique des commentaires et formulaire d’ajout.

Interventions

Date	Duree (min)	Description
2026-07-02 18:06:45.672286	45	Problème résolu après remplacement du routeur

Affectations

Technicien	Date	Actif	Commentaire
Aucune affectation.			

FIGURE 13.7 – Historique des interventions et affectations.

Nouvel incident

Reference

Titre

Description

Priorite

Categorie

Créer l'incident

FIGURE 13.8 – Formulaire de création d’un incident.

Date	Operation	Table	Utilisateur PostgreSQL	Detail
2026-07-04 01:17:32.537882	UPDATE	incident	postgres	Nouvelle : {"id_incident":15,"reference":"INC-DEMO-001","titre":"Imprimante RH hors service","description":"Aucune impression possible depuis ce matin","date_ouverture":"2026-07-04T01:02:18.333608","date_limite_resolution":null,"date_cloture":"2026-07-04T01:17:32.537882","id_utilisateur":3,"id_priorite":1,"id_statut":null,"id_categorie":2} Ancienne : {"id_incident":15,"reference":"INC-DEMO-001","titre":"Imprimante RH hors service","description":"Aucune impression possible depuis ce matin","date_ouverture":"2026-07-04T01:02:18.333608","date_limite_resolution":null,"date_cloture":null,"id_utilisateur":3,"id_priorite":1,"id_statut":1,"id_categorie":2}
2026-07-04 01:02:18.333608	INSERT	incident	postgres	Nouvelle : {"id_incident":15,"reference":"INC-DEMO-001","titre":"Imprimante RH hors service","description":"Aucune impression possible depuis ce matin","date_ouverture":"2026-07-04T01:02:18.333608","date_limite_resolution":null,"date_cloture":null,"id_utilisateur":3,"id_priorite":1,"id_statut":1,"id_categorie":2}

FIGURE 13.9 – Journal d’audit (ADMIN) : détail JSON avant/après.

Utilisateurs					
+ Nouvel utilisateur					
Nom	Email	Role	Service	Actif	
Souleymane Ba	souleymaneba@gmail.com	TECHNICIEN	None	Oui	Modifier Desactiver
Aminata Diallo	aminatadiallo@gmail.com	ADMIN	None	Oui	Modifier Desactiver
Aissatou Gaye	aissatougaye@gmail.com	UTILISATEUR	None	Oui	Modifier Desactiver

FIGURE 13.10 – Gestion des utilisateurs (ADMIN).

Le menu affiche l’intégralité des liens disponibles pour ce rôle : Tableau de bord, Incidents, Nouvel incident, Journal d’audit, Utilisateurs. La page de détail regroupe les informations générales, le bloc « Actions technicien / admin », puis l’historique complet.

Vue TECHNICIEN

Gestion Incidents

Tableau de bord

Incidents

Souleymane Ba (TECHNICIEN)

Deconnexion

Connecte en tant que TECHNICIEN

Tableau de bord

Statistiques par statut

Statut	Nombre d'incidents
OUVERT	1

Derniers incidents ouverts

Reference	Titre	Priorite	Date ouverture
INC001	Panne réseau corrigée	Faible	2026-07-02 17:53:04.214036

FIGURE 13.11 – Tableau de bord (TECHNICIEN) : menu réduit.

INC001 - Panne réseau corrigée

Statut : OUVERT

Priorité : Faible

Catégorie : Réseau

Déclare par : Souleymane Ba

Ouvert le : 2026-07-02 17:53:04.214036

Description :
Internet indisponible

Actions technicien / admin

FIGURE 13.12 – Détail d'un incident (TECHNICIEN).

Actions technicien / admin

Affecter a un technicien

Souleymane Ba

Commentaire d'affectation

Affecter

Changer le statut

OUVERT

Mettre a jour le statut

Nouvelle intervention

FIGURE 13.13 – Bloc Actions technicien/admin, présent pour ce rôle.

Le menu affiche uniquement Tableau de bord et Incidents – ni Nouvel incident, ni Journal d’audit, ni Utilisateurs ne sont visibles. Le bloc « Actions technicien / admin » reste présent sur le détail d’un incident, cohérent avec le fait que ce rôle est habilité à traiter les incidents.

Vue UTILISATEUR

Gestion Incidents
Tableau de bord
Incidents
Nouvel incident
Aissatou Gaye (UTILISATEUR)
Deconnexion

Connecte en tant que UTILISATEUR

Tableau de bord

Statistiques par statut

Statut	Nombre d'incidents
OUVERT	1

Derniers incidents ouverts

Reference	Titre	Priorite	Date ouverture
INC001	Panne réseau corrigée	Faible	2026-07-02 17:53:04.214036

FIGURE 13.14 – Tableau de bord (UTILISATEUR) : menu spécifique.

Nouvel incident

Reference

INC-2026-001

Titre

Description

Priorite

Faible

Categorie

Logiciel

Creer l'incident

FIGURE 13.15 – Formulaire Nouvel incident (accessible à ce rôle).

INC001 - Panne réseau corrigée

Statut : OUVERT

Priorite : Faible

Categorie : Réseau

Declare par : Souleymane Ba

Ouvert le : 2026-07-02 17:53:04.214036

Description : Internet indisponible

FIGURE 13.16 – Détail d'un incident (UTILISATEUR) : aucun bloc d'action.

Commentaires

Auteur	Date	Contenu
Aucun commentaire.		

Ajouter un commentaire

Please fill out this field.

Envoyer

FIGURE 13.17 – Commentaires (ajout possible).

Interventions

Date	Duree (min)	Description
2026-07-02 18:06:45.672286	45	Problème résolu après remplacement du routeur

Affectations

Technicien	Date	Actif	Commentaire
Aucune affectation.			

FIGURE 13.18 – Interventions/affectations en lecture seule.

Le menu affiche Tableau de bord, Incidents et Nouvel incident – ce rôle a accès à la création d’un incident, contrairement au TECHNICIEN. Sur la page de détail, aucun bloc d’action n’apparaît : la page passe directement des informations générales aux commentaires (ajout possible), puis aux interventions et affectations, affichées uniquement en lecture.

Un correctif de privilèges découvert pendant les tests

En testant la page de détail avec le compte UTILISATEUR, une erreur `psycopg2.errors.InsufficientPrivilege` est apparue : la table `utilisateurs` n’accordait `SELECT` qu’à `admin_role` et `technicien_role`, alors que la page de détail en a besoin pour afficher le nom du déclarant. Le correctif :

```
GRANT SELECT ON utilisateurs TO utilisateur_role;
```

Ce test illustre concrètement l’intérêt du contrôle d’accès à deux niveaux (chapitre 12) : une page mal protégée côté privilèges PostgreSQL déclenche une erreur explicite plutôt qu’une fuite silencieuse de données, ce qui a permis de détecter et corriger l’oubli.

Tableau récapitulatif des pages par rôle

Page	Route	UTIL.	TECH.	ADMIN
Connexion	/login	Oui	Oui	Oui
Tableau de bord	/dashboard	Oui	Oui	Oui
Liste des incidents	/incidents	Oui	Oui	Oui
Nouvel incident	/incidents/nouveau	Oui	Non	Oui
Détail d’un incident	/incidents/<id>	Oui (lecture)	Oui	Oui
Affecter / statut / intervention / clôture	/incidents/<id>/...	Non	Oui	Oui
Ajouter un commentaire	/incidents/<id>/commentaire	Non	Oui	Oui
Journal d’audit	/audit	Non	Non	Oui
Gestion des utilisateurs	/admin/utilisateurs	Non	Non	Oui

TABLE 13.1 – Pages accessibles selon le rôle de l’utilisateur connecté.

Chapitre 14

Résultats obtenus

1. Une base de données PostgreSQL complète : 9 tables, contraintes d'intégrité, 3 vues, 6 fonctions/procédures, 3 triggers.
2. Cinq rôles PostgreSQL (`postgres`, `app_login`, `admin_role`, `technicien_role`, `utilisateur_role`) avec des privilèges différenciés et cohérents avec les besoins métier de chaque profil.
3. Un mécanisme d'audit automatique traçant toutes les modifications sur les incidents, vérifié avec des données réelles.
4. Trois transactions démonstratives illustrant `COMMIT`, `ROLLBACK` et `SAVEPOINT`, exécutées et vérifiées.
5. Une application web Flask fonctionnelle, avec authentification sécurisée (mots de passe hashés avec `scrypt`) et un affichage adapté dynamiquement au rôle de l'utilisateur connecté, vérifié par des captures réelles pour les 3 rôles.
6. Un correctif de privilèges identifié et corrigé en cours de test (`GRANT` manquant pour `utilisateur_role` sur la table `utilisateurs`), démontrant l'efficacité du contrôle d'accès à deux niveaux.

Chapitre 15

Difficultés rencontrées et solutions

Difficulté	Solution apportée
Erreur <code>psycpg2.OperationalError: fe_sendauth</code> (mot de passe non transmis)	Vérification étape par étape des variables d'environnement (<code>.env</code>), confirmation que le mot de passe du rôle PostgreSQL correspondait bien à celui attendu par l'application.
Erreur <code>database does not exist</code> (faute de frappe sur le nom de la base : <code>informatique/informatiques</code>)	Vérification systématique du nom exact de la base dans les commandes et dans <code>.env</code> .
Mots de passe stockés en clair dans le jeu de données de test	Migration des mots de passe existants vers un hash sécurisé avec <code>generate_password_hash()</code> , vérifié avec <code>check_password_hash()</code> à la connexion.
Privilèges manquants sur certaines tables/vues pour <code>technicien_role</code> et <code>utilisateur_role</code> (affectation, commentaire, intervention, les 3 vues, et la table utilisateurs)	Ajout de <code>GRANT SELECT</code> complémentaires dans <code>roles_privileges.sql</code> , découverts progressivement en testant chaque rôle.
Rôles métier (<code>admin_role</code> , <code>technicien_role</code> , <code>utilisateur_role</code>) créés initialement avec <code>LOGIN</code> au lieu de <code>NOLOGIN</code>	Correction avec <code>ALTER ROLE ... NOLOGIN</code> , sans impact sur le fonctionnement puisque <code>SET ROLE</code> ne nécessite pas que le rôle cible ait <code>LOGIN</code> .
Cohérence entre le nom des blueprints Flask et les <code>url_for()</code> utilisés dans les templates (<code>auth_bp</code> vs <code>auth</code>)	Harmonisation du nom du blueprint d'authentification en <code>'auth'</code> partout dans le code.
Blueprint <code>incidents.py</code> initialement dupliquait le code de <code>auth.py</code> au lieu de contenir la logique métier	Réécriture complète du blueprint avec toutes les routes nécessaires (dashboard, liste, création, détail, actions, audit).

TABLE 15.1 – Difficultés rencontrées durant le projet et solutions apportées.

Conclusion

Ce projet a permis de mettre en pratique l'ensemble des mécanismes avancés de PostgreSQL exigés par le cahier des charges : modélisation, contraintes, vues, fonctions et procédures PL/pgSQL, triggers, transactions, gestion des rôles et des privilèges, et audit. L'intégration avec une interface Flask a également permis de démontrer concrètement l'application de ces mécanismes à travers un cas d'usage réaliste : la gestion des incidents informatiques, du signalement par un utilisateur jusqu'à la clôture par un technicien, avec une traçabilité complète de toutes les étapes. Les tests menés avec les trois rôles réels ont également permis d'identifier et de corriger plusieurs oublis de privilèges, renforçant la cohérence entre la sécurité documentée et la sécurité réellement appliquée.

Annexe A – Comptes de démonstration

Rôle	Email	Mot de passe
ADMIN	aminatadiallo@gmail.com	motdepasse123
TECHNICIEN	souleymane@gmail.com	motdepasse456
UTILISATEUR	aissatougaye@gmail.com	motdepasse789

TABLE 17.1 – Comptes de démonstration utilisés pour les tests.

Annexe B – Contraintes d’intégrité complètes (extrait)

```
ALTER TABLE incident ADD CONSTRAINT incident_pkey PRIMARY KEY
(id_incident);
ALTER TABLE incident ADD CONSTRAINT incident_reference_key UNIQUE
(reference);
ALTER TABLE incident ADD CONSTRAINT fk_incident_user
FOREIGN KEY (id_utilisateur) REFERENCES utilisateurs(id_utilisateur);
ALTER TABLE incident ADD CONSTRAINT fk_incident_priorite
FOREIGN KEY (id_priorite) REFERENCES priorite(id_priorite);
ALTER TABLE incident ADD CONSTRAINT fk_incident_statut
FOREIGN KEY (id_statut) REFERENCES statut(id_statut);
ALTER TABLE incident ADD CONSTRAINT fk_incident_categorie
FOREIGN KEY (id_categorie) REFERENCES categorie(id_categorie);
ALTER TABLE utilisateurs ADD CONSTRAINT utilisateurs_email_key UNIQUE
(email);
ALTER TABLE utilisateurs ADD CONSTRAINT utilisateurs_pkey PRIMARY KEY
(id_utilisateur);
```

Annexe C – Script complet des rôles et privilèges (roles_privileges)

```

CREATE ROLE app_login LOGIN PASSWORD '...';
CREATE ROLE admin_role NOLOGIN;
CREATE ROLE technicien_role NOLOGIN;
CREATE ROLE utilisateur_role NOLOGIN;
GRANT admin_role, technicien_role, utilisateur_role TO app_login
WITH INHERIT OPTION, SET OPTION;
GRANT EXECUTE ON FUNCTION creer_incident(...) TO admin_role,
utilisateur_role;
GRANT EXECUTE ON FUNCTION affecter_technicien(...) TO admin_role,
technicien_role;
GRANT EXECUTE ON FUNCTION changer_statut_incident(...) TO admin_role,
technicien_role;
GRANT EXECUTE ON PROCEDURE ajouter_commentaire(...)
TO admin_role, technicien_role, utilisateur_role;
GRANT EXECUTE ON PROCEDURE ajouter_intervention(...) TO admin_role,
technicien_role;
GRANT EXECUTE ON PROCEDURE cloturer_incident(...) TO admin_role,
technicien_role;
— Corrections apportées suite aux tests avec les 3 rôles réels
GRANT SELECT ON affectation TO technicien_role, utilisateur_role;
GRANT SELECT ON commentaire TO technicien_role, utilisateur_role;
GRANT SELECT ON intervention TO technicien_role, utilisateur_role;
GRANT SELECT ON incidents_ouverts TO technicien_role, utilisateur_role;
GRANT SELECT ON statistiques_incidents TO technicien_role,
utilisateur_role;
GRANT SELECT ON suivi_incidents TO technicien_role;
GRANT SELECT ON utilisateurs TO utilisateur_role;

```

Annexe D – Fichiers livrés avec ce rapport

1. `script_examen_BDD.sql` – script complet de création de la base (tables, contraintes, vues, fonctions, triggers, données de test).
2. `sql/roles_privileges.sql` – création des rôles PostgreSQL et attribution des privilèges.
3. `sql/transactions_demo.sql` – 3 transactions démonstratives (COMMIT/ROLLBACK/SAVEPOINT).
4. Code source complet de l'application Flask (dossier `app/`).

Annexe E – Commandes de mise en route

```

# 1. Créer la base
psql -U postgres -c "CREATE DATABASE
gestion_des_incidents_informatiques"
# 2. Importer le schema complet

```



```
psql -U postgres -d gestion_des_incidents_informatiques -f
script_examen_BDD.sql
# 3. Creer les roles et privileges
psql -U postgres -d gestion_des_incidents_informatiques -f
sql/roles_privileges.sql
# 4. Installer les dependances Python
pip install -r requirements.txt
# 5. Configurer .env (DB_HOST, DB_NAME, DB_USER, DB_PASSWORD,
FLASK_SECRET_KEY)
# 6. Lancer l'application
python run.py
```